

SIMATSENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO4:** Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. (BL3, BL4, BL5)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Technical Presentation

DURATION: 2 HRS
(25 Marks)

1. Dynamic Programming for Sequential Decision-Making in Reinforcement Learning

AIM:-

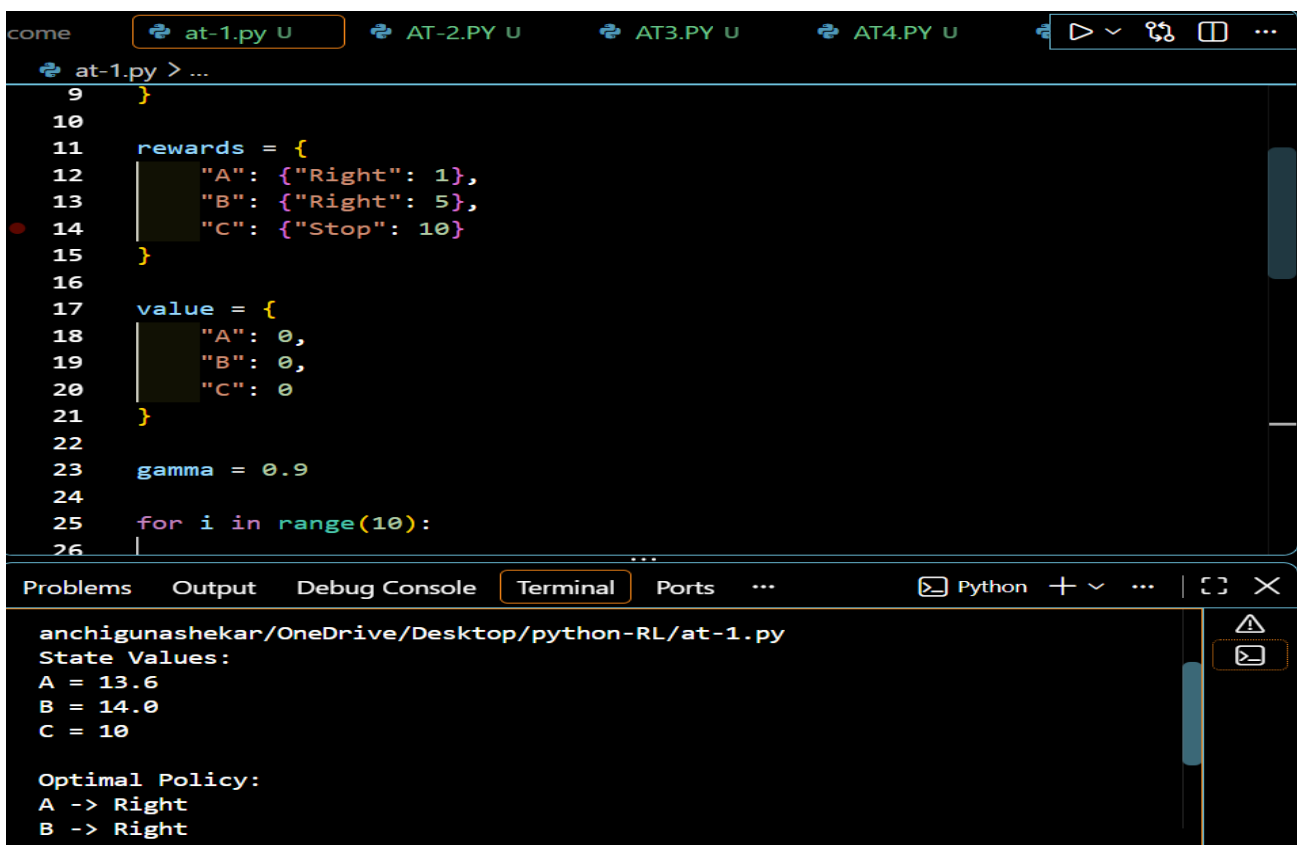
To implement Dynamic Programming techniques for sequential decision-making in Reinforcement Learning and understand the Bellman Principle of Optimality, Policy Evaluation, Policy Improvement, Value Iteration, and Policy Iteration.

ALGORITHM:-

1. Start the program.
2. Define the states and available actions.

3. Define the transition probabilities.
4. Define the reward for each state-action pair.
5. Initialize the value function.
6. Apply the Bellman equation.
7. Perform policy evaluation.
8. Perform policy improvement.
9. Repeat until the policy becomes stable.

OUTPUT:-



The screenshot shows a code editor with a file named `at-1.py` open. The code defines a reinforcement learning environment with three states (A, B, C) and two actions (Left, Right). The rewards are defined as follows:

```
rewards = {
    "A": {"Right": 1},
    "B": {"Right": 5},
    "C": {"Stop": 10}
}
```

The value function is initialized to zero for all states:

```
value = {
    "A": 0,
    "B": 0,
    "C": 0
}
```

The discount factor γ is set to 0.9. The code runs a loop for 10 iterations.

The terminal output shows the state values and the optimal policy:

```
anchigunashekar/OneDrive/Desktop/python-RL/at-1.py
State Values:
A = 13.6
B = 14.0
C = 10

Optimal Policy:
A -> Right
B -> Right
```

INDUSTRIAL APPLICATIONS:-

- Robot path planning
- Autonomous vehicles
- Inventory management
- Manufacturing scheduling
- Resource allocation

CONCLUSION:-

Thus, Dynamic Programming was implemented using Value Iteration. The Bellman equation was used to calculate state values and determine the optimal policy. Dynamic Programming is useful for decision-making problems where the environment model is known.

2. Monte Carlo Methods and Temporal-Difference Learning

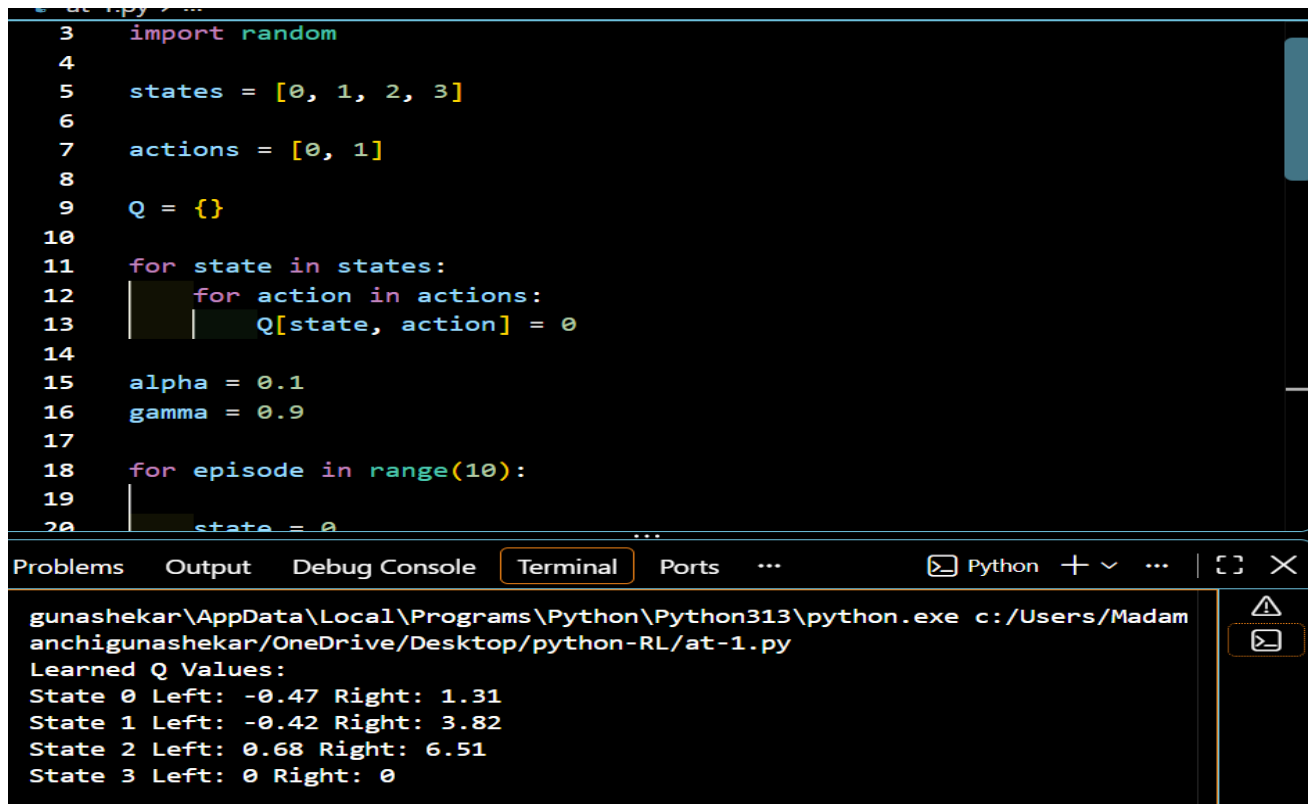
AIM

To study and compare Monte Carlo, TD(0), SARSA, and Q-Learning methods based on learning, sample efficiency, and performance.

ALGORITHM

1. Start the program.
2. Define states and actions.
3. Initialize value or Q-value tables.
4. Generate an episode.
5. Calculate rewards.
6. Apply Monte Carlo learning.
7. Apply TD(0) learning.
8. Apply SARSA.
9. Apply Q-Learning.
10. Compare the learned values.

OUTPUT:-



```
3 import random
4
5 states = [0, 1, 2, 3]
6
7 actions = [0, 1]
8
9 Q = {}
10
11 for state in states:
12     for action in actions:
13         Q[state, action] = 0
14
15 alpha = 0.1
16 gamma = 0.9
17
18 for episode in range(10):
19     state = 0
20     ...
```

gunashekar\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Madam anchigunashekar/OneDrive/Desktop/python-RL/at-1.py

Learned Q Values:

State	Action	Q-Value
State 0	Left	-0.47
State 0	Right	1.31
State 1	Left	-0.42
State 1	Right	3.82
State 2	Left	0.68
State 2	Right	6.51
State 3	Left	0
State 3	Right	0

CONCLUSION

Thus, Monte Carlo and Temporal-Difference learning methods were studied. TD methods can update values before an episode finishes, making them more sample efficient in many sequential decision-making problems. Q-Learning learns the optimal action by using the maximum next-state Q-value.

3. DeepQ-Networks for Intelligent Decision-Making

AIM

To implement a simple DeepQ-Network (DQN) and understand its architecture, Experience Replay, and Target Network for intelligent decision-making.

ALGORITHM

1. Start the program.
2. Create the environment.
3. Define the state and action spaces.
4. Create the neural network.
5. Select an action using ϵ -greedy.
6. Store experience in replay memory.
7. Sample experiences from replay memory.
8. Train the Q-network.
9. Update the target network.
10. Repeat training.

OUTPUT:-

```
24     [3]
25 ], dtype=np.float32)
26
27 # Target Q-values
28 targets = np.array([
29     [1, 0],
30     [2, 0],
31     [3, 0],
32     [0, 5]
33 ], dtype=np.float32)
34
35 # Train the model
36 model.fit(
37     states,
38     targets,
39     epochs=20,
40     verbose=0
41 )
```

...
Problems 1 Output Debug Console Terminal Ports ... Python + v ... | {} >
C:\Users\Madamanchigunashekar\OneDrive\Desktop\python-RL>C:\Users\Madamanchigunashekar\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Madamanchigunashekar/OneDrive/Desktop/python-RL/at-1.py
Learned Q Values:
State 0 Left: -0.47 Right: 1.31
State 1 Left: -0.42 Right: 3.82
State 2 Left: 0.68 Right: 6.51
State 3 Left: 0 Right: 0

CONCLUSION

Thus, a simple DQN was implemented using a neural network. The DQN estimates Q-values for different actions and selects the action with the highest value. Experience Replay and Target Networks help improve training stability.

4. Comparative Analysis of Advanced Deep Q-Learning Algorithms

AIM

To study and compare **Double DQN**, **Dueling DQN**, and **Prioritized Experience Replay (PER)** with standard DQN for intelligent decision-making.

ALGORITHM

1. Start the program.
2. Create the environment.
3. Define states and actions.
4. Implement standard DQN.
5. Implement Double DQN.
6. Implement Dueling DQN.
7. Implement Prioritized Experience Replay.
8. Train the algorithms using the same environment.
9. Record rewards and training progress.

OUTPUT:-

```
1  # Simple comparison of DQN variants
2
3  algorithms = [
4      "DQN",
5      "Double DQN",
6      "Dueling DQN",
7      "Prioritized Replay"
8  ]
9
10 rewards = [60, 70, 75, 80]
11
12 print("Algorithm Performance\n")
13
14 for i in range(len(algorithms)):
15     print(algorithms[i], "Reward:", rewards[i])
16
17 best_algorithm = algorithms.index(max(rewards))
18
```

Debug Console (Ctrl+Shift+Y)

Problems Output Debug Console Terminal Ports ... Python + v ...

anchigunashekar/OneDrive/Desktop/python-RL/at-1.py

Algorithm Performance

DQN Reward: 60
Double DQN Reward: 70
Dueling DQN Reward: 75
Prioritized Replay Reward: 80

Best Algorithm: Prioritized Replay

CONCLUSION

Thus, advanced DQN algorithms were compared. Double DQN reduces Q-value overestimation, Dueling DQN provides better value estimation, and Prioritized Experience Replay focuses training on important experiences. These improvements can provide better learning performance than standard DQN.

5. Policy-Based Reinforcement Learning for Continuous Control

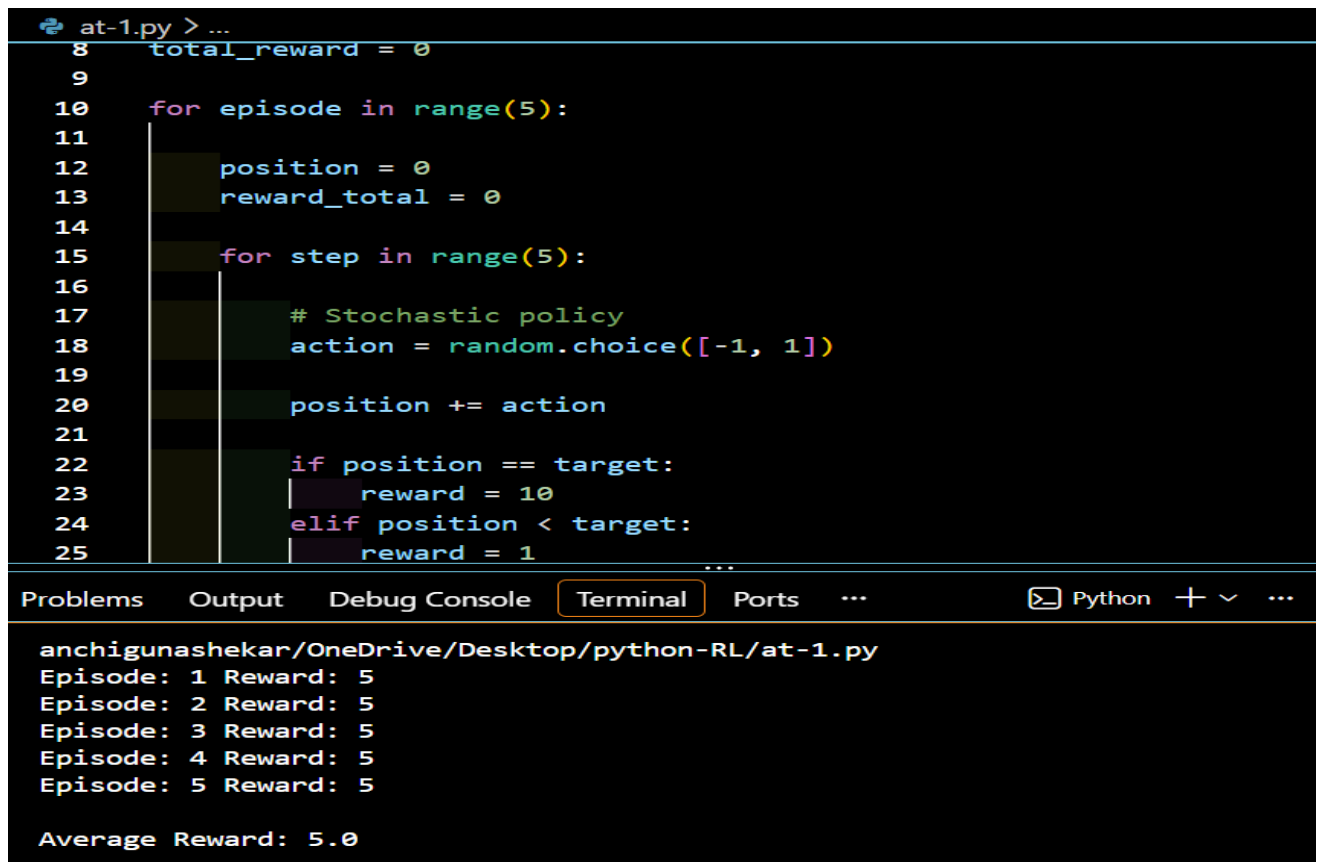
AIM

To implement a simple **Policy Gradient** and **REINFORCE** model and understand stochastic policy search for continuous control problems.

ALGORITHM

1. Start the program.
2. Define the environment.
3. Define the state space.
4. Define the action space.
5. Create a stochastic policy.
6. Select actions from the policy.
7. Collect rewards.
8. Calculate the return.
9. Update the policy using REINFORCE.
10. Repeat for multiple episodes.

OUTPUT:-



```
at-1.py > ...
8 total_reward = 0
9
10 for episode in range(5):
11     position = 0
12     reward_total = 0
13
14     for step in range(5):
15         # Stochastic policy
16         action = random.choice([-1, 1])
17         position += action
18
19         if position == target:
20             reward = 10
21         elif position < target:
22             reward = 1
23
24     ...
25
```

Problems Output Debug Console Terminal Ports ... Python + v ...

anchigunashekar/OneDrive/Desktop/python-RL/at-1.py

Episode: 1 Reward: 5
Episode: 2 Reward: 5
Episode: 3 Reward: 5
Episode: 4 Reward: 5
Episode: 5 Reward: 5

Average Reward: 5.0

CONCLUSION

Thus, a simple Policy Gradient and REINFORCE approach was implemented for continuous control. The stochastic policy allows the agent to explore different actions and improve its policy using received rewards. Policy-based methods are especially useful for problems with continuous or large action spaces.

Code of Conduct:

I E Teja Vardhan Reddy (192425297) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

E TEJAVARDHAN REDDY

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism		Exemplary professionalism	Professional conduct with	Basic professionalism	Lacks professionalism; minimal

m	10	m, leadership, and ethical awareness	effective team collaboration	; uneven team participation	contribution or ethical awareness
---	----	--	---------------------------------	-----------------------------------	---